

《暗狱回廊：逐暗旅人》课程设计报告 --- ## 一、项目概述 ### 1.1 项目背景 本项目是基于 Qt 框架开发的回合制地牢 RPG 游戏。玩家将扮演一名勇敢的逐暗旅人，在黑暗的地牢中探索未知、击败怪物、收集装备，最终挑战强大的 BOSS。 ### 1.2 开发环境 | 项目 | 版本 | |-----|-----| | 操作系统 | Windows 11 | | 开发框架 | Qt 6.11.1 | | 编程语言 | C++17 | | 编译器 | MinGW 64-bit | | 开发工具 | Qt Creator 14.0.0 | ### 1.3 项目目标 - 实现完整的回合制战斗系统 - 设计随机地牢生成机制 - 开发角色养成与装备系统 - 实现多存档与自动保存功能 - 提供良好的用户交互体验 --- ## 二、功能需求分析 ### 2.1 功能模块划分 根据需求文档，本项目分为以下核心模块： | 模块 | 功能描述 | |-----|-----| | 主菜单模块 | 开始游戏、读取存档、删除存档、游戏说明、退出游戏 | | 地牢探索模块 | 地图显示、角色移动、房间交互、迷雾系统 | | 战斗系统模块 | 回合制战斗、攻击/技能/道具/逃跑操作、战斗日志 | | 角色养成模块 | 属性管理、升级系统、技能解锁 | | 道具装备模块 | 背包管理、装备穿戴、道具使用 | | 商店系统模块 | 商品购买、装备购买、商品刷新 | | 存档系统模块 | 多存档管理、自动保存、进度读取 | ### 2.2 用例分析 ##### 2.2.1 玩家用例 | 用例名称 | 描述 | 前置条件 | |-----|-----|-----| | 开始新游戏 | 创建新存档并命名角色 | 主菜单界面 | | 读取存档 | 选择已有存档继续游戏 | 存在至少一个存档 | | 删除存档 | 删除指定存档 | 存在至少一个存档 | | 移动角色 | 通过方向按钮移动角色 | 地牢探索界面 | | 触发战斗 | 进入战斗房间自动触发战斗 | 角色位于战斗房间 | | 打开宝箱 | 获得随机道具或装备 | 角色位于宝箱房间 | | 使用道具 | 消耗道具获得效果 | 背包中有可用道具 | | 穿戴装备 | 装备武器/防具/饰品 | 背包中有可用装备 | | 购买商品 | 消耗金币购买道具或装备 | 商店界面，金币足够 | | 保存游戏 | 自动保存到当前层开始房间 | 角色到达新楼层 | --- ## 三、系统架构设计 ### 3.1 整体架构 本项目采用 MVC (Model-View-Controller) 架构模式，将数据模型、界面视图和控制逻辑分离：



Model 层				
Character	Dungeon	Item	Equipment	Skill
SaveManager	Shop			

3.2 核心类设计 #### 3.2.1 Character 类 (角色类) | 属性 | 类型 | 说明
| |-----|-----|-----| | m_name | QString | 角色名称 | | m_maxHp | int | 最大生命值 | | m_hp | int | 当前生命值 | | m_maxMp | int | 最大魔力值 | | m_mp | int | 当前魔力值 | | m_attack | int | 攻击力 | | m_defense | int | 防御力 | | m_level | int | 等级 | | m_experience | int | 经验值 | | m_gold | int | 金币 | | 方法 | 说明 | |-----|-----| | takeDamage(int damage) | 受到伤害 | | heal(int amount) | 恢复生命值 | | addMp(int amount) | 恢复魔力值 | | levelUp() | 升级 | | learnSkill(Skill* skill) | 学习技能 | | equipItem(Equipment* equip) | 装备物品 | | unequipItem(EquipmentType type) | 卸下装备 | #### 3.2.2 Dungeon 类 (地牢类) | 属性 | 类型 | 说明 | |-----|-----|-----| | m_floor | int | 当前层数 | | m_rooms | Room[5][5] | 房间二维数组 | | m_playerX, m_playerY | int | 玩家坐标 | | 方法 | 说明 | |-----|-----| | generateFloor() | 生成随机楼层 | | getRoom(int x, int y) | 获取指定房间 | | hasNextFloor() | 是否有下一层 | | nextFloor() | 进入下一层 | #### 3.2.3 BattleWindow 类 (战斗窗口) | 属性 | 类型 | 说明 | |-----|-----|-----| | m_player | Character* | 玩家角色 | | m_enemy | Character* | 敌人角色 | | m_isPlayerTurn | bool | 是否玩家回合 | | m_battleLog | QTextEdit* | 战斗日志 | | 方法 | 说明 | |-----|-----| | startBattle() | 开始战斗 | | playerAttack() | 玩家攻击 | | playerSkill(int index) | 玩家使用技能 | | playerItem(int index) | 玩家使用道具 | | playerEscape() | 玩家逃跑 | | enemyTurn() | 敌人回合 | | checkBattleEnd() | 检查战斗结束 | #### 3.2.4 SaveManager 类 (存档管理) | 方法 | 说明 | |-----|-----| | saveGame(...) | 保存游戏进度 | | loadGame(...) | 加载游戏进度 | | getSaveInfoList() | 获取存档列表 | | deleteSave(int index) | 删除存档 | | isSaveNameExists(QString name) | 检查存档名是否存在 | --- ## 四、核心模块实现 ### 4.1 主菜单模块 主菜单采用 QWidget 容器，包含游戏标题和功能按钮：

```
cpp
void MainWindow::showMainMenu()
{
    // 创建主菜单容器
    m_mainMenuWidget = new QWidget(this);
    QVBoxLayout* layout = new QVBoxLayout(m_mainMenuWidget);
```

```

        // 添加游戏标题
        QLabel* titleLabel = new QLabel("<h1>暗狱回廊：逐暗旅人
</h1>", m_mainMenuWidget);
        titleLabel->setAlignment(Qt::AlignCenter);
        layout->addWidget(titleLabel);

        // 添加功能按钮
        QStringList buttonNames = {"开始新游戏", "读取存档", "删除存
档", "游戏说明", "退出游戏"};
        for (const QString& name : buttonNames) {
            QPushButton* btn = new QPushButton(name,
m_mainMenuWidget);
            layout->addWidget(btn);
            // 连接信号槽
            connect(btn, &QPushButton::clicked, this, [this,
name]() {
                handleMainMenuAction(name);
            });
        }

        setCentralWidget(m_mainMenuWidget);
    }

```

4.2 地牢探索模块 地牢地图采用 5x5 的网格布局，使用 QGridLayout 实现：

```

cpp
void MainWindow::updateDungeonMap()
{
    // 清空现有地图
    QLayoutItem* item;
    while ((item = m_mapLayout->takeAt(0)) != nullptr) {
        delete item->widget();
        delete item;
    }

    // 绘制地图格子
    for (int y = 0; y < 5; ++y) {
        for (int x = 0; x < 5; ++x) {
            QLabel* cell = new QLabel();
            cell->setFixedSize(62, 62);
            cell->setStyleSheet("border: 1px solid #333;");

            // 检查是否已探索
            if (!m_explored[y][x]) {
                // 显示迷雾
                QPixmap fog(":/images/fog.png");
                cell->setPixmap(fog.scaled(62, 62));
            } else {

```

```

        Room room = m_dungeon->getRoom(x, y);
        // 根据房间类型显示图片
        switch (room.type) {
            case RoomType::ENTRANCE:
                // 入口房间
                break;
            case RoomType::BATTLE:

cell->setPixmap(QPixmap(":/images/battle.png").scaled(62,
62));

                break;
            case RoomType::TREASURE:

cell->setPixmap(QPixmap(":/images/treasure.png").scaled(62,
62));

                break;
            case RoomType::BOSS:

cell->setPixmap(QPixmap(":/images/boss.png").scaled(62, 62));
                break;
        }

        // 显示玩家位置
        if (x == m_playerX && y == m_playerY) {
            QPixmap player(":/images/player.png");
            cell->setPixmap(player.scaled(62, 62));
        }
    }

    m_mapLayout->addWidget(cell, y, x);
}
}
}

```

4.3 回合制战斗系统 战斗系统的核心逻辑基于回合顺序判定:

```

cpp
void BattleWindow::startBattle()
{
    m_isPlayerTurn = m_player->speed() >= m_enemy->speed();
    updateTurnLabel();

    if (m_isPlayerTurn) {
        addBattleLog("你的回合!");
    } else {
        addBattleLog("敌人的回合!");
        QTimer::singleShot(1000, this,
&BattleWindow::enemyTurn);
    }
}

```

```

void BattleWindow::playerAttack()
{
    int damage = std::max(1, m_player->attack() -
m_enemy->defense());
    m_enemy->takeDamage(damage);
    addBattleLog(QString("你对 %1 造成了 %2 点伤害！
").arg(m_enemy->name()).arg(damage));
    updateEnemyStats();

    if (checkBattleEnd()) return;

    m_isPlayerTurn = false;
    updateTurnLabel();
    QTimer::singleShot(1000, this, &BattleWindow::enemyTurn);
}

void BattleWindow::enemyTurn()
{
    int damage = std::max(1, m_enemy->attack() -
m_player->defense());
    m_player->takeDamage(damage);
    addBattleLog(QString("%1 对你造成了 %2 点伤害！
").arg(m_enemy->name()).arg(damage));
    updatePlayerStats();

    if (checkBattleEnd()) return;

    m_isPlayerTurn = true;
    updateTurnLabel();
    addBattleLog("你的回合！");
}

```

4.4 角色养成系统 角色升级时自动提升属性并解锁技能:

```

cpp
void Character::levelUp()
{
    m_level++;
    m_maxHp += 20;
    m_hp = m_maxHp;
    m_maxMp += 10;
    m_mp = m_maxMp;
    m_attack += 3;
    m_defense += 2;
    m_experience = 0;

    // 解锁新技能
    if (m_level == 3) {
        // 解锁技能 1
    }
}

```

```

    } else if (m_level == 5) {
        // 解锁技能 2
    } else if (m_level == 7) {
        // 解锁技能 3
    }
}

```

4.5 存档系统 存档系统使用 JSON 格式存储游戏数据:

```

cpp
bool SaveManager::saveGame(int index, const QString& name,
Character* player,
                                int floor, QVector<Item*>& items,
                                QVector<Equipment*>& equipped,
                                QVector<Equipment*>& storage,
                                QVector<Skill*>& skills, int x, int
y)
{
    QJsonObject saveData;
    saveData["name"] = name;
    saveData["floor"] = floor;
    saveData["playerX"] = x;
    saveData["playerY"] = y;

    // 保存玩家属性
    QJsonObject playerData;
    playerData["name"] = player->name();
    playerData["level"] = player->level();
    playerData["hp"] = player->hp();
    playerData["maxHp"] = player->maxHp();
    playerData["mp"] = player->mp();
    playerData["maxMp"] = player->maxMp();
    playerData["attack"] = player->attack();
    playerData["defense"] = player->defense();
    playerData["gold"] = player->gold();
    playerData["experience"] = player->experience();
    saveData["player"] = playerData;

    // 保存道具、装备、技能...

    QJsonDocument doc(saveData);
    QFile file(getSaveFilePath(index));
    if (file.open(QIODevice::WriteOnly)) {
        file.write(doc.toJson());
        return true;
    }
    return false;
}

```

--- ## 五、关键技术实现 ### 5.1 Qt 信号槽机制 本项目广泛使用 Qt 的信号槽机制实现组件间通信:

```
cpp
// 按钮点击事件
connect(m_attackButton, &QPushButton::clicked, this,
&BattleWindow::playerAttack);

// 战斗结束信号
connect(m_battleWindow, &BattleWindow::battleWon, this,
&MainWindow::on_battleWon);
connect(m_battleWindow, &BattleWindow::battleLost, this,
&MainWindow::on_battleLost);

// 属性变更信号
connect(m_player, &Character::hpChanged, this,
&MainWindow::updatePlayerStats);
```

5.2 随机地牢生成算法 地牢生成采用伪随机数生成器, 确保每层布局不同:

```
cpp
void Dungeon::generateFloor()
{
    // 初始化所有房间为空白
    for (int y = 0; y < 5; ++y) {
        for (int x = 0; x < 5; ++x) {
            m_rooms[y][x].type = RoomType::EMPTY;
            m_rooms[y][x].cleared = false;
        }
    }

    // 设置入口 (0,0)
    m_rooms[0][0].type = RoomType::ENTRANCE;
    m_rooms[0][0].cleared = true;

    // 设置 BOSS 房 (4,4)
    m_rooms[4][4].type = RoomType::BOSS;

    // 随机生成战斗房和宝箱房
    int battleCount = 8 +
QRandomGenerator::global()->bounded(4);
    int treasureCount = 2 +
QRandomGenerator::global()->bounded(2);

    while (battleCount > 0) {
        int x = QRandomGenerator::global()->bounded(5);
        int y = QRandomGenerator::global()->bounded(5);
        if (m_rooms[y][x].type == RoomType::EMPTY) {
            m_rooms[y][x].type = RoomType::BATTLE;
```

```

        battleCount--;
    }
}

while (treasureCount > 0) {
    int x = QRandomGenerator::global()->bounded(5);
    int y = QRandomGenerator::global()->bounded(5);
    if (m_rooms[y][x].type == RoomType::EMPTY) {
        m_rooms[y][x].type = RoomType::TREASURE;
        treasureCount--;
    }
}
}

```

5.3 迷雾系统 迷雾系统实现了探索机制，玩家周围 9 格可见：

```

cpp
void MainWindow::updateExplored()
{
    // 标记玩家周围 9 格为已探索
    for (int dy = -1; dy <= 1; ++dy) {
        for (int dx = -1; dx <= 1; ++dx) {
            int nx = m_playerX + dx;
            int ny = m_playerY + dy;
            if (nx >= 0 && nx < 5 && ny >= 0 && ny < 5) {
                m_explored[ny][nx] = true;
            }
        }
    }

    // BOSS 房始终可见
    m_explored[4][4] = true;

    updateDungeonMap();
}

```

--- ## 六、运行说明 ### 6.1 编译步骤 1. 打开 Qt Creator，选择「打开项目」 2. 选择 `dungeon-rpg.pro` 文件 3. 配置构建工具为 MinGW 64-bit 4. 点击构建按钮（锤子图标） ### 6.2 运行步骤 1. 点击运行按钮（绿色三角形） 2. 在主菜单选择「开始新游戏」 3. 输入存档名称和角色名称 4. 开始游戏 ### 6.3 游戏操作 | 操作 | 说明 | |-----|-----| | 方向按钮 | 移动角色 | | 使用道具 | 打开背包使用道具 | | 穿戴装备 | 装备武器/防具/饰品 | | 查看技能 | 查看已解锁技能 | | 进入商店 | 购买商品 | | 进入下一层 | 击败 BOSS 后进入下一层 | --- ## 七、总结 ### 7.1 完成的功能 本项目成功实现了以下核心功能： 1. **主菜单界面**：支持开始游戏、读取存档、删除存档、游戏说明、退出游戏 2. **地牢探索系统**：5x5 随机地图、迷雾系统、房间交互 3. **回合制战斗系统**：攻击、技能、道

具、逃跑操作 4. **角色养成系统**：升级、技能解锁、属性提升 5. **道具装备系统**：背包管理、装备穿戴 6. **商店系统**：商品购买、商品刷新 7. **存档系统**：多存档管理、自动保存 ### 7.2 技术亮点 1. 采用 MVC 架构，代码结构清晰 2. 使用 Qt 信号槽机制实现组件解耦 3. JSON 格式存储存档，便于数据管理 4. 随机地牢生成算法保证游戏可玩性 5. 迷雾系统增加探索乐趣 ### 7.3 待改进之处 1. 增加音效系统提升游戏体验 2. 增加更多怪物类型和技能 3. 优化界面美化效果 4. 添加更多装备和道具种类 5. 增加成就系统 --- **项目地址**：
https://gitee.com/xiaolu_xlll/dungeon-rpg **完成日期**：2026 年 7 月